CEE6501 — Lecture 12.2

Force Density Method

Learning Objectives

By the end of this lecture, you will be able to:

- explain the basic idea behind the force density method for form-finding
- define branch-node matrices and interpret how they describe network connectivity
- derive the linear equilibrium equations used in the force density method
- solve for unknown free-node coordinates when force densities and support locations are prescribed
- connect the force density method to the broader form-finding framework of Veenendaal and Block

Agenda

- Part 1 - A Single Framework for Form-Finding Methods
- Part 2 - Intro to Force Density
- Part 3 - Branch-Node Matrix Formulation
- Part 4 - Derivation of the Force Density Method
- Part 5 - Worked Example
- Wrap-Up

Part 1 - A Single Framework for Form-Finding Methods

High-Level Motivation

In many **form-active structures**, the final geometry is not known in advance.

Instead, the shape must be found as part of the analysis, because **geometry and internal force depend on one another**.

This is the central idea behind **form finding**.

Typical examples include:

- cable nets
- tensioned membranes
- pneumatic surfaces
- gridshells
- other lightweight systems whose shape depends on the flow of forces

References For This Part Of The Lecture

For this introductory discussion, we will draw mainly on two papers by Veenendaal and Block.

1. A conference paper published in 2011 at the IASS conference: *A Framework for Comparing Form Finding Methods*
2. A more developed journal paper published in 2012 in the International Journal of Solids and Structures: *An overview and comparison of structural form finding methods for general networks*

The journal paper is more comprehensive, but for today's lecture the conference paper is a very good starting point.

Links to both papers are posted on GitHub and saved in the Lecture 12 folder.

International Journal of Solids and Structures 49 (2012) 3741–3753

Contents lists available at SciVerse ScienceDirect

International Journal of Solids and Structures

journal homepage: www.elsevier.com/locate/ijsolstr

An overview and comparison of structural form finding methods for general networks

D. Veenendaal *, P. Block

Institute of Technology in Architecture, ETH Zurich, Wolfgang-Pauli-Strasse 15, 8093 Zurich, Switzerland

ARTICLE INFO

Article history:
Received 31 May 2012
Received in revised form 13 July 2012
Available online 18 August 2012

Keywords:
Form finding
Shape finding
Force density
Dynamic relaxation
Updated reference strategy
Tension structures

ABSTRACT

This paper discusses and compares existing form finding methods for discrete networks. Well-known methods such as the force density method, dynamic relaxation, updated reference strategy and others are discussed by mathematically structuring and presenting them in the same way, using the same notation and combining terminology. Based on this, a single computational framework using a sparse branch-node data structure is presented. It is shown how each method approaches the initial equilibrium problem, defines and linearizes the equilibrium equations applied to linear elements, and uses particular solving strategies. This framework marginalizes any differences related to operating platforms, programming language and style, offering a better baseline for independent comparison of performance and results. As a consequence, it is possible to more clearly relate, distinguish and compare existing methods, allow for hybrid methods and identify new avenues for research.

© 2012 Elsevier Ltd. All rights reserved.

Historical Context

Veenendaal and Block begin by placing form finding in a broader historical context.

By the 1960s and 1970s, computational form-finding had become important for lightweight spanning systems such as:

- cable-net roofs
- membrane roofs
- pneumatic structures
- gridshells

As these structural systems developed, several different numerical strategies also emerged for finding equilibrium shapes.

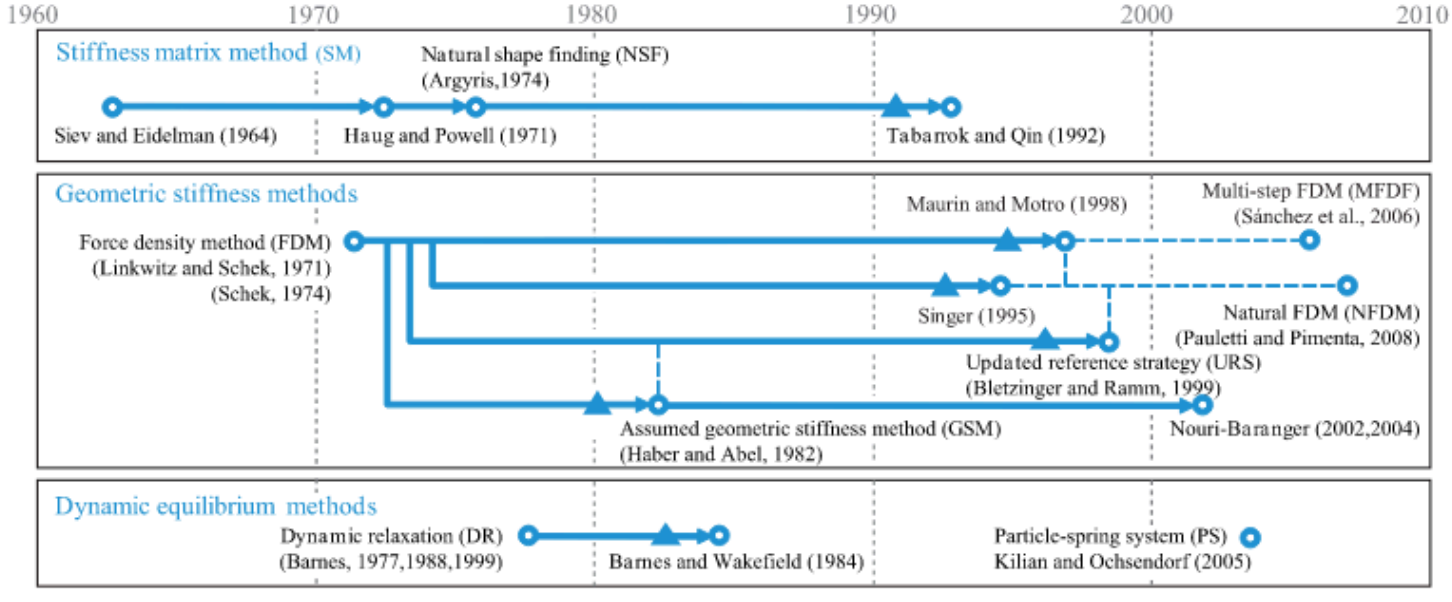
Three Main Families Of Methods

Veenendaal and Block group classical form-finding methods into three main families:

- **Force density methods**
- **Dynamic relaxation methods**
- **Stiffness methods**

These methods are often presented separately, but they are all trying to solve the same basic problem:

- describe a structural network
- satisfy equilibrium
- determine the corresponding geometry


1960
1970
1980
1990
2000
2010
Stiffness matrix method (SM)
Natural shape finding (NSF)
(Argyris,1974)
Siev and Eidelman (1964)
Haug and Powell (1971)
Tabarrok and Qin (1992)
Geometric stiffness methods
Maurin and Motro (1998)
Multi-step FDM (MFDF)
(Sánchez et al., 2006)
Force density method (FDM)
(Linkwitz and Schek, 1971)
(Schek, 1974)
Singer (1995)
Natural FDM (NFDM)
(Pauletti and Pimenta, 2008)
Updated reference strategy (URS)
(Bletzinger and Ramm, 1999)
Assumed geometric stiffness method (GSM)
(Haber and Abel, 1982)
Nouri-Baranger (2002,2004)
Dynamic equilibrium methods
Dynamic relaxation (DR)
(Barnes, 1977,1988,1999)
Barnes and Wakefield (1984)
Particle-spring system (PS)
Kilian and Ochsendorf (2005)

Types Of Structural Systems

These methods are applied to:

- **Bar-and-joint frameworks**: axial elements connected at nodes
- **Triangulated surface or shell meshes**: planar elements connected through nodes

In this course, we will focus only on the simpler **bar-and-joint case**.

That includes structures such as:

- trusses
- cable nets
- simple space-frame type networks

What Distinguishes These Methods

Force density methods use the ratio of force to length as the central quantity.

Dynamic relaxation methods use an analogy with motion, where residual forces are converted into nodal accelerations and the structure iteratively moves toward equilibrium.

Stiffness methods use structural stiffness directly, so the form is found by enforcing equilibrium with an explicit stiffness model.

Why A Common Framework Is Useful

A key message of the Veenendaal and Block papers is that these methods are often **more similar than they first appear**.

What changes from one method to another is mainly:

- how internal forces are represented
- how the unknown geometry is updated
- what quantity is chosen as the primary variable

So rather than treating them as completely separate ideas, Veenendaal and Block show that they can be compared within a single mathematical viewpoint.

A Preview Of The Unified Framework

These three method families can be written within a common iterative framework.

A representative expression is:

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \left(\mathbf{C}^T\mathbf{K}\mathbf{C}\right)^{-1}\left(\mathbf{p} - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}_f\,\mathbf{x}_f - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}\,\mathbf{x}_i\right)$$

At this stage, do **not** worry about every symbol.

The main idea is that all three methods can be viewed as:

- writing equilibrium on a network of nodes and branches (i.e., members or elements)
- evaluating the current internal force state
- updating the nodal coordinates toward equilibrium

The Unified Framework Conceptually

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \left(\mathbf{C}^T\mathbf{K}\mathbf{C}\right)^{-1}\left(\mathbf{p} - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}_f\,\mathbf{x}_f - \mathbf{C}^T\mathbf{L}^{-1}\mathbf{F}\mathbf{C}\,\mathbf{x}_i\right)$$

This looks like a standard iterative correction:

$$\text{new coordinates} = \text{current coordinates} + \text{correction based on the residual}$$

We will return later to the meaning and usage of the full equation.

For now, the main takeaway is that the methods differ less in overall structure, and more in **how the internal branch quantities are defined**.

Force Over Length Term

One important quantity that appears in the unified framework is

$$\mathbf{L}^{-1}\mathbf{F}$$

which, branch-by-branch, is simply **force divided by length**.

For a single branch, this is

$$\frac{f_i}{l_i}$$

Importance of the Force Over Length Term

This ratio matters because, in axial networks, the branch force is always tied to the **current geometry** through the branch length.

So if the shape changes, the lengths change, and therefore the relation between force and geometry changes as well.

That is exactly why form finding is difficult in the first place: **equilibrium depends on geometry, and geometry affects the branch force state**.

The quantity force-over-length is important because it captures that branch force state in a form that works naturally with the network equations.

Transition To Force Density

Among the three method families, the **force density method** is especially attractive because it leads to a particularly clean mathematical formulation that leads to a linear solution.

Its central idea is to use, for each branch,

$$q_i = \frac{f_i}{l_i}$$

as the primary analysis parameter.

The quantity q_i is called the **force density** of branch i.

So before returning later to the broader unified framework, we first need to understand the **force density method on its own terms**, beginning with Schek's original formulation

Part 2 - Intro to Force Density

> *Schek, The Force Density Method for Form Finding and Computation of General Networks (1973)*

The Basic Problem

Schek starts from a very direct question:

Given a **pin-jointed network** of cables or bars, what equilibrium shapes can that network take under given supports and loading?

Geometry is Unknown

At the start, the **connectivity** of the network is known, but the **final geometry is not**.

So the task is not simply to analyze a structure of known shape.

The task is to **find the shape itself**.

The essential requirement is that the final configuration must satisfy **equilibrium**, meaning that the sum of forces must be zero at every node.

Why Form Matters

For long-span and lightweight structures, the structural form has a major influence on:

- the internal forces
- the deformations
- the overall structural behavior

In many cases, equilibrium final forms are not available in advance.

So form finding becomes a necessary step before more detailed structural analysis can even begin.

Network View Of The Structure

Schek models the structure as a **network of nodes and branches**.

- nodes are joints
- branches are axial members such as cables or bars
- the branches connect only through the nodes
- the geometry is allowed to change until an equilibrium shape is found

Why A New Method Was Helpful

Earlier approaches often relied on:

- physical experiments
- trial-and-error procedures
- nonlinear iterative calculations with many steps

Those approaches could be useful, but they were not always efficient when exploring many possible shapes.

Schek's goal was to build a computational method that keeps the network description simple, while making the equilibrium calculation much more direct.

The New Description Parameter

Schek's key idea is to describe each branch using the ratio of force to length:

$$q_i = \frac{s_i}{l_i}$$

where:

- s_i = axial force in branch i
- l_i = length of branch i
- q_i = force density of branch i

So instead of prescribing the final shape directly, we prescribe one scalar quantity for each branch: its **force density**.

note: following Schek's notation in the paper, so $s_i = f_i$

The Solution Becomes Linear

This choice is powerful because, when the force densities are treated as known, the equilibrium equations become **linear in the unknown nodal coordinates**.

That is one of the main reasons the method became so influential in form finding.

So the classical force density method works like this:

- the network connectivity is known
- the support locations are known
- the force densities are prescribed (set by the designer)
- the unknown node coordinates are solved from a linear system

This is the main practical advantage emphasized by Schek.

Linear Form Finding Versus Nonlinear Constraints

Schek also makes an important qualification.

The linear force density formulation is very effective for finding an equilibrium shape, but it does **not** automatically enforce every other design requirement.

If we add further conditions, the problem may become nonlinear again.

Examples include:

- geometric constraints
- prescribed mesh conditions
- additional force constraints
- other design restrictions beyond simple equilibrium

The Force Density method is linear in its basic form, but it can also be extended into a **nonlinear method** when extra constraints are imposed.

Why The Linear Version Is Still Valuable

Even when additional constraints are needed later, the linear method is still extremely useful.

It gives:

- a fast first equilibrium shape
- a clean way to explore many different forms
- a good starting point for more detailed nonlinear calculations

So the linear force density method is both a useful form-finding tool on its own and a practical starting point for more advanced analysis.

Notation And Diagonal Matrices

Schek uses a compact matrix notation throughout the paper.

The basic convention is:

- **lowercase letters** are vectors
- **uppercase letters** are matrices

A particularly important pattern is when the **same letter appears in both lowercase and uppercase**.

In that case, the uppercase version is usually the **diagonal matrix formed from the vector**.

For example, if

$$\mathbf{q}^T = \begin{bmatrix} q_1 & q_2 & \cdots & q_m \end{bmatrix}$$

then

$$\mathbf{Q} = \begin{bmatrix} q_1 & 0 & 0 & \cdots & 0 \\ 0 & q_2 & 0 & \cdots & 0 \\ 0 & 0 & q_3 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & q_m \end{bmatrix}$$

Similarly:

- $\mathbf{l}$ becomes $\mathbf{L}$
- $\mathbf{s}$ becomes $\mathbf{S}$

Setting up the Force Density Method

The next step is to describe the **connectivity of the network** using the **branch-node matrix** does.

It tells us which nodes each branch connects, and with what orientation.

Once that connectivity is written in matrix form, the branch quantities (i.e, force densities) and the nodal coordinates can be linked together in a very compact way.

So next, we move to the **branch-node matrix formulation**.

Part 3 - Branch-Node Matrix Formulation

Connectivity Comes First

In the direct stiffness method, we usually work at the **element level**:

- define local element behavior
- transform to global coordinates
- assemble into a global system

In the force density method, we shift viewpoint.

We work more directly with:

- **nodes**
- **branches**
- the **topology** of how nodes are connected

The key object is the **branch-node matrix**.

Definition Of The Branch-Node Matrix

The branch-node matrix, $\mathbf{C}_s$, is an $m \times n$ matrix, where:

- m = number of branches
- n = number of nodes

So in general, it is **not square**.

More specifically, we write

$$\mathbf{C}_s \in \mathbb{R}^{m \times n}$$

Each **row** corresponds to one branch, and each **column** corresponds to one node.

For a branch i connecting start node j to end node k, the entries of the i-th row are

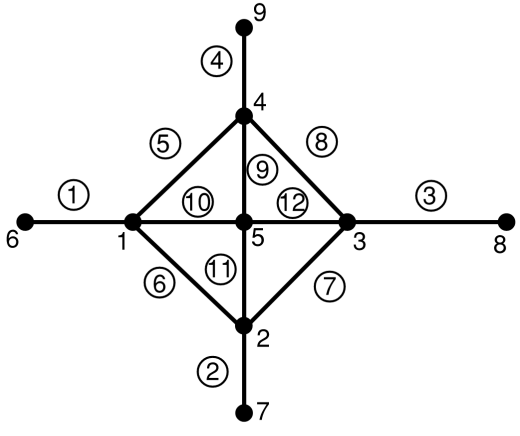
$$C_s(i,r) = \begin{cases} +1 & \text{if } r = j \\ -1 & \text{if } r = k \\ 0 & \text{otherwise} \end{cases}$$

So each row records:

- **which two nodes** the branch connects
- the **chosen orientation** of that branch

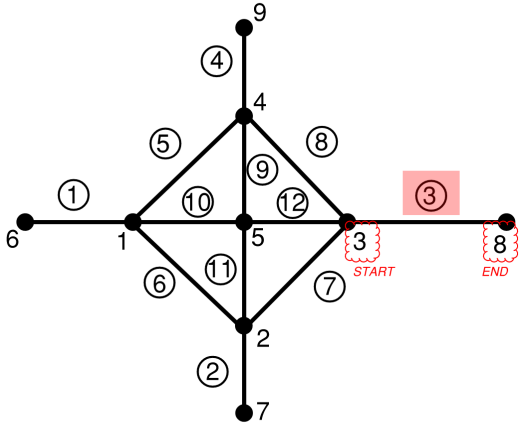
The orientation is just a bookkeeping convention, but it matters because it determines the sign of the branch coordinate differences.

Example Of A Branch-Node Matrix



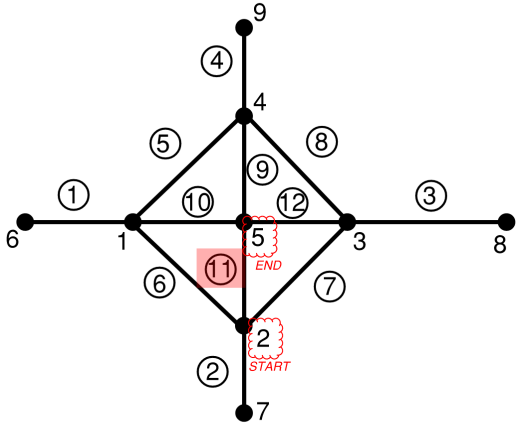
$$\mathbf{C}_s = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Example Of A Branch-Node Matrix



$$\mathbf{C}_s = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & \boxed{1} & 0 & 0 & 0 & 0 & \boxed{-1} & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Example Of A Branch-Node Matrix



$$\mathbf{C}_s = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & \boxed{1} & 0 & 0 & \boxed{-1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Why The Branch-Node Matrix Is Useful

The branch-node matrix is useful because it gives us a compact way to represent the **connectivity of the network** inside the algebra.

Once the connectivity is written in matrix form, we no longer have to describe each branch-node relation one at a time.

Instead, the full network can be handled through matrix operations.

This is powerful because the same matrix can then be used to:

- encode which nodes each branch connects
- keep track of branch orientation through sign
- move between **nodal quantities** and **branch-wise quantities**
- assemble branch contributions back into nodal equilibrium equations

So the branch-node matrix is more than just a definition of connectivity.

It is the main matrix tool that lets the topology of the network participate directly in the formulation.

Branch Coordinate Differences

Using the branch-node matrix we can also easily calculate coordinate differences between nodal pairs

If $\mathbf{x}$ is the vector of all nodal x-coordinates, then the branch coordinate differences in the x-direction are

$$\mathbf{u} = \mathbf{C}\mathbf{x}$$

which is analogous to doing $\Delta_x = x_{start} - x_{end}$ for each node pair defining the branch

Likewise, in the y- and z-directions,

$$\mathbf{v} = \mathbf{C}\mathbf{y}$$

$$\mathbf{w} = \mathbf{C}\mathbf{z}$$

Partition into Free And Fixed Nodes

You can further partitions the nodes into:

- **free nodes** with unknown coordinates
- **fixed nodes** with prescribed coordinates

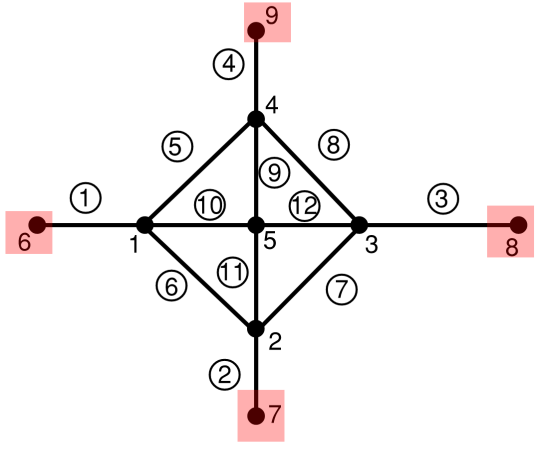
Accordingly, the full branch-node matrix is partitioned as

$$\mathbf{C}_s = \begin{bmatrix} \mathbf{C} & \mathbf{C}_f \end{bmatrix}$$

where:

- $\mathbf{C}$ acts on the unknown free-node coordinates
- $\mathbf{C}_f$ acts on the known fixed-node coordinates

Partitioned Branch-Node Matrix



$$\mathbf{C}_s = [\mathbf{C} \mid \mathbf{C}_f] = \left[\begin{array}{ccccc:cccc} 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & -1 \\ 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 \end{array}\right]$$

Branch Coordinate Differences (Partitioned)

If $\mathbf{x}$ is the vector of unknown free-node x-coordinates and $\mathbf{x}_f$ is the vector of fixed-node x -coordinates, then the branch coordinate differences in the x-direction are

$$\mathbf{u} = \mathbf{C}\mathbf{x} + \mathbf{C}_f\mathbf{x}_f$$

$$\underbrace{\mathbf{u}}_{m\times 1} = \underbrace{\mathbf{C}}_{m\times n_f}\underbrace{\mathbf{x}}_{n_f\times 1} + \underbrace{\mathbf{C}_f}_{m\times n_r}\underbrace{\mathbf{x}_f}_{n_r\times 1}$$

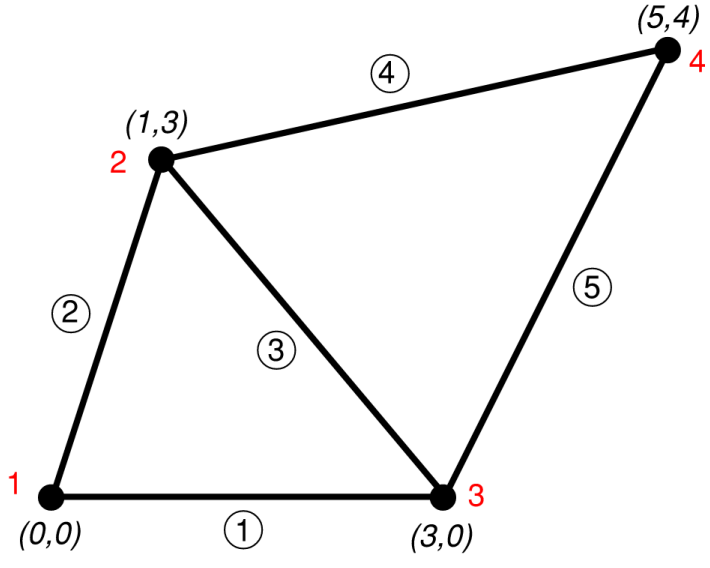
Likewise, in the y- and z-directions,

$$\mathbf{v} = \mathbf{C}\mathbf{y} + \mathbf{C}_f\mathbf{y}_f$$

$$\mathbf{w} = \mathbf{C}\mathbf{z} + \mathbf{C}_f\mathbf{z}_f$$

So these vectors contain, branch-by-branch, the signed coordinate differences that define each element in each direction.

Example Calculation Of Branch Projections



$$\mathbf{C} = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & -1 \end{bmatrix}$$

$$\mathbf{x}^T = \begin{bmatrix} 0 & 1 & 3 & 5 \end{bmatrix} \qquad \mathbf{y}^T = \begin{bmatrix} 0 & 3 & 0 & 4 \end{bmatrix}$$

$$\mathbf{Cx} = \begin{bmatrix} (1)(0)+(0)(1)+(-1)(3)+(0)(5) \\ (1)(0)+(-1)(1)+(0)(3)+(0)(5) \\ (0)(0)+(1)(1)+(-1)(3)+(0)(5) \\ (0)(0)+(1)(1)+(0)(3)+(-1)(5) \\ (0)(0)+(0)(1)+(1)(3)+(-1)(5) \end{bmatrix} = \begin{bmatrix} -3 \\ -1 \\ -2 \\ -4 \\ -2 \end{bmatrix}$$

In [10]:

```python
import numpy as np

# ------------------------------------------------------------
# Problem data
# ------------------------------------------------------------

branches = [
    (1, 3),
    (1, 2),
    (2, 3),
    (2, 4),
    (3, 4),
]

fixed_nodes = [1, 2]
free_nodes = [3, 4]

# Node coordinates from the figure
coords = {
    1: (0.0, 0.0),
    2: (1.0, 3.0),
    3: (3.0, 0.0),
    4: (5.0, 4.0),
}
```

In [11]:

```
# ------------------------------------------------------------
# Build partitioned branch-node matrices
# ------------------------------------------------------------
node_order = free_nodes + fixed_nodes
node_index = {node: i for i, node in enumerate(node_order)}

m = len(branches)
n_free = len(free_nodes)
n_fixed = len(fixed_nodes)

C_s = np.zeros((m, n_free + n_fixed), dtype=int)

for r, (start, end) in enumerate(branches):
    C_s[r, node_index[start]] = 1
    C_s[r, node_index[end]] = -1

C = C_s[:, :n_free]
C_f = C_s[:, n_free:]
```

In [12]:
```
print("\nC_s =\n", C_s)
print("\nC =\n", C)
print("\nC_f =\n", C_f)
```

```
C_s =
 [[-1  0  1  0]
 [ 0  0  1 -1]
 [-1  0  0  1]
 [ 0 -1  0  1]
 [ 1 -1  0  0]]

C =
 [[-1  0]
 [ 0  0]
 [-1  0]
 [ 0 -1]
 [ 1 -1]]

C_f =
 [[ 1  0]
 [ 1 -1]
 [ 0  1]
 [ 0  1]
 [ 0  0]]
```

In [13]:

```
# ------------------------------------------------------------
# Coordinate vectors
# ------------------------------------------------------------
x = np.array([coords[node][0] for node in free_nodes], dtype=float).reshape(-1
y = np.array([coords[node][1] for node in free_nodes], dtype=float).reshape(-1

x_f = np.array([coords[node][0] for node in fixed_nodes], dtype=float).reshape
y_f = np.array([coords[node][1] for node in fixed_nodes], dtype=float).reshape

# ------------------------------------------------------------
# Branch coordinate differences
# u = delta_x, v = delta_y
# based on the chosen branch orientation
# ------------------------------------------------------------
u = C @ x + C_f @ x_f
v = C @ y + C_f @ y_f
```

In [14]:

```
# ------------------------------------------------------------
# Print results
# ------------------------------------------------------------
np.set_printoptions(precision=3, suppress=True)

print("\nu = Δx =\n", u)
print("\nv = Δy =\n", v)
```

```
u = Δx =
 [[-3.]
 [-1.]
 [-2.]
 [-4.]
 [-2.]]

v = Δy =
 [[ 0.]
 [-3.]
 [ 3.]
 [-1.]
 [-4.]]
```

Interpreting The Matrix

Suppose the free-node coordinates in the x-direction are

$$\mathbf{x} = \begin{bmatrix} x_3 \\ x_4 \end{bmatrix}$$

and the fixed-node coordinates are

$$\mathbf{x}_f = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Then

$$\mathbf{u} = \mathbf{C}\mathbf{x} + \mathbf{C}_f\mathbf{x}_f$$

returns one number per branch:

the signed difference in the x-direction between the two nodes of that branch.

So:

- nodal coordinates live at **points**
- multiplying by $\mathbf{C}$ converts them into **branch projections**

In [15]:

```
# Choose coordinates and verify the branch projection relation

x_free = np.array([2.0, 4.0])   # x3, x4
x_fixed = np.array([0.0, 6.0])  # x1, x2

u = C @ x_free + C_f @ x_fixed
print("Branch x-projections u =\n", u)

# Direct check from geometry
direct_u = []
coords_x = {1: 0.0, 2: 6.0, 3: 2.0, 4: 4.0}
for j, k in branches:
    direct_u.append(coords_x[j] - coords_x[k])

print("\nDirect geometric check =\n", np.array(direct_u))
```

```
Branch x-projections u =
 [-2. -6.  4.  2. -2.]

Direct geometric check =
 [-2. -6.  4.  2. -2.]
```

What Does $\mathbf{C}^T\mathbf{Q}\mathbf{C}$ Do?

This is one of the most important operations in the method.

It helps to read it from **right to left**.

First, $\mathbf{C}$ takes nodal coordinates and converts them into **branch coordinate differences**.

Then, $\mathbf{Q}$ weights each branch difference by its corresponding **force density**.

Finally, $\mathbf{C}^T$ gathers those weighted branch contributions back to the **nodes**.

So, at a high level,

$$\text{nodes} \xrightarrow{\mathbf{C}} \text{branches} \xrightarrow{\mathbf{Q}} \text{weighted branches} \xrightarrow{\mathbf{C}^T} \text{nodes again}$$

This is why the operation $\mathbf{C}^T\mathbf{Q}\mathbf{C}$ produces a nodal matrix.

It takes branch information, weights it, and assembles its effect back onto the nodal system.

Why This Feels Familiar

There is a useful analogy here with the direct stiffness method.

In DSM, we often think in terms of:

- element behavior
- transformation
- assembly

Here, something similar is happening in a more compact network form:

- $\mathbf{C}$ maps nodal data to branch differences
- $\mathbf{Q}$ applies the branch properties
- $\mathbf{C}^T$ assembles the branch effects back to the nodes

So while $\mathbf{C}$ is not a standard coordinate transformation matrix, the overall operation plays a somewhat similar structural role in the algebra.

Transition To The Derivation

Now that we have a matrix description of the network connectivity, we have the missing ingredient needed to write equilibrium in compact form.

The next step is to combine:

- the branch-node matrix
- the branch projections
- the force densities

to derive the governing equilibrium equations of the force density method.

So next, we move from **network description** to the **actual derivation of the method**.

Part 4 - Derivation Of The Force Density Method

Part 4 - Derivation of the Force Density Method

step 1 - define branch projections and branch lengths

For each branch, define its projected differences in the coordinate directions:

$$\mathbf{u} = \mathbf{C}\mathbf{x} + \mathbf{C}_f\mathbf{x}_f$$

$$\mathbf{v} = \mathbf{C}\mathbf{y} + \mathbf{C}_f\mathbf{y}_f$$

$$\mathbf{w} = \mathbf{C}\mathbf{z} + \mathbf{C}_f\mathbf{z}_f$$

For a planar problem, we usually only need
$mathbfu$
and
$mathbfv$
.

The branch lengths are collected in the vector

$$\mathbf{l} = \begin{bmatrix} l_1 \\ l_2 \\ \vdots \\ l_m \end{bmatrix}$$

with

$$l_i = \sqrt{u_i^2 + v_i^2 + w_i^2}$$

Schek then introduces diagonal matrices built from these vectors, such as $mathbfU$

,
$mathbfV$

$mathbfW$

and
$mathbfL$

!
$mathbfs$

step 2 - nodal equilibrium written with branch forces

Let s_i be the axial force in branch i.

If we collect all branch forces into the vector

, then the nodal loads in each coordinate direction can be written as

$$\mathbf{C}^T\mathbf{U}\mathbf{L}^{-1}\mathbf{s} = \mathbf{p}_x$$

$$\mathbf{C}^T\mathbf{V}\mathbf{L}^{-1}\mathbf{s} = \mathbf{p}_y$$

$$\mathbf{C}^T\mathbf{W}\mathbf{L}^{-1}\mathbf{s} = \mathbf{p}_z$$

This is still not especially convenient, because the branch lengths are unknown and appear nonlinearly.

So the next step is the decisive one.

step 3 - define force density

Define the force density vector by

$$\mathbf{q} = \mathbf{L}^{-1}\mathbf{s}$$

or componentwise,

$$q_i = \frac{s_i}{l_i}$$

Now use the identities

$$\mathbf{U}\mathbf{q} = \mathbf{Q}\mathbf{u}, \qquad \mathbf{V}\mathbf{q} = \mathbf{Q}\mathbf{v}, \qquad \mathbf{W}\mathbf{q} = \mathbf{Q}\mathbf{w}$$

where

$$\mathbf{Q} = \mathrm{diag}(q_1, q_2, \ldots, q_m)$$

Substituting into equilibrium gives

$$\mathbf{C}^T\mathbf{Q}\mathbf{u} = \mathbf{p}_x$$

$$\mathbf{C}^T\mathbf{Q}\mathbf{v} = \mathbf{p}_y$$

$$\mathbf{C}^T\mathbf{Q}\mathbf{w} = \mathbf{p}_z$$

This is the key simplification.

step 4 - substitute the coordinate differences

Insert

$$\mathbf{u} = \mathbf{C}\mathbf{x} + \mathbf{C}_f\mathbf{x}_f$$

into the equilibrium equation:

$$\mathbf{C}^T\mathbf{Q}\left(\mathbf{C}\mathbf{x} + \mathbf{C}_f\mathbf{x}_f\right) = \mathbf{p}_x$$

Rearrange:

$$\mathbf{C}^T\mathbf{Q}\mathbf{C}\,\mathbf{x} = \mathbf{p}_x - \mathbf{C}^T\mathbf{Q}\mathbf{C}_f\,\mathbf{x}_f$$

Define

$$\mathbf{D} = \mathbf{C}^T\mathbf{Q}\mathbf{C}, \qquad \mathbf{D}_f = \mathbf{C}^T\mathbf{Q}\mathbf{C}_f$$

Then the force density equations become

$$\mathbf{D}\mathbf{x} = \mathbf{p}_x - \mathbf{D}_f\mathbf{x}_f$$

$$\mathbf{D}\mathbf{y} = \mathbf{p}_y - \mathbf{D}_f\mathbf{y}_f$$

$$\mathbf{D}\mathbf{z} = \mathbf{p}_z - \mathbf{D}_f\mathbf{z}_f$$

These are **linear equations in the unknown coordinates**.

why this is such a big deal

For prescribed:

- connectivity
- support coordinates
- nodal loads
- force densities

the unknown free-node coordinates follow from a **single linear solve**.

That is the classical appeal of the force density method.

In a planar case, once we solve

$$\mathbf{x} = \mathbf{D}^{-1}\left(\mathbf{p}_x - \mathbf{D}_f\mathbf{x}_f\right)$$

and

$$\mathbf{y} = \mathbf{D}^{-1}\left(\mathbf{p}_y - \mathbf{D}_f\mathbf{y}_f\right)$$

we can reconstruct the geometry, compute the branch lengths, and then recover branch forces from

$$\mathbf{s} = \mathbf{L}\mathbf{q}$$

So the method is "one-step" in the sense that the geometry solve itself is linear once $mathbfq$

is specified.

In [16]:

```
# Solve a small planar force-density example

# Fixed node coordinates
xy_fixed = {
    1: np.array([0.0, 0.0]),
    2: np.array([6.0, 0.0]),
}

# Applied loads at free nodes
# Only vertical loads in this example
p_x = np.array([0.0, 0.0])
p_y = np.array([-1.0, -1.0])

# Prescribed force densities for each branch in the same order as 'branches'
q = np.array([1.0, 1.0, 1.0, 1.0, 0.5])
Q = np.diag(q)

# Fixed coordinate vectors in [node1, node2] order
x_f = np.array([xy_fixed[1][0], xy_fixed[2][0]])
y_f = np.array([xy_fixed[1][1], xy_fixed[2][1]])

D = C.T @ Q @ C
D_f = C.T @ Q @ C_f

x = np.linalg.solve(D, p_x - D_f @ x_f)
y = np.linalg.solve(D, p_y - D_f @ y_f)

print("D =\n", D)
print("\nD_f =\n", D_f)
```

```python
print("\nSolved free-node x-coordinates =", x)
print("Solved free-node y-coordinates =", y)
```

```
D =
 [[ 2.5 -0.5]
 [-0.5  1.5]]

D_f =
 [[-1. -1.]
 [ 0. -1.]]

Solved free-node x-coordinates = [3.429 5.143]
Solved free-node y-coordinates = [-0.571 -0.857]
```

Part 5 - Worked Example

hand-solution structure

For a simple planar example with two free nodes, the workflow is:

explicit equations for the teaching example

1. choose the connectivity and build $mathbfC$ and $mathbfC_f$
2. prescribe the force densities and build $mathbfQ$
3. compute

$$\mathbf{C} = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{C}_f = \begin{bmatrix} -1 & 0 \\ 0 & -1 \\ -1 & 0 \\ 0 & -1 \\ 0 & 0 \end{bmatrix}$$

and

$$\mathbf{D} = \mathbf{C}^T\mathbf{Q}\mathbf{C}$$

and with

$$\mathbf{D}_f = \mathbf{C}^T\mathbf{Q}\mathbf{C}_f$$

4. solve separately for the unknown x- and y-coordinates
5. recover branch lengths and then branch forces

$$\mathbf{q} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0.5 \end{bmatrix}$$

This is exactly the same logic as the paper, just written in a more classroom-friendly sequence.

we obtain

$$\mathbf{Q} = \mathrm{diag}(1, 1, 1, 1, 0.5)$$

Then

$$\mathbf{D} = \mathbf{C}^T\mathbf{Q}\mathbf{C} = \begin{bmatrix} 2.5 & -0.5 \\ -0.5 & 2.5 \end{bmatrix}$$

and

$$\mathbf{D}_f = \mathbf{C}^T\mathbf{Q}\mathbf{C}_f = \begin{bmatrix} -1 & -1 \\ -1 & -1 \end{bmatrix}$$

Since the supports lie at $(0,0)$ and $(6,0)$, we have

$$\mathbf{x}_f = \begin{bmatrix} 0 \\ 6 \end{bmatrix}, \qquad \mathbf{y}_f = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

and for downward nodal loads

$$\mathbf{p}_x = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad \mathbf{p}_y = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

the coordinate solves are

$$\mathbf{D}\mathbf{x} = \mathbf{p}_x - \mathbf{D}_f\mathbf{x}_f = \begin{bmatrix} 6 \\ 6 \end{bmatrix}$$

$$\mathbf{D}\mathbf{y} = \mathbf{p}_y - \mathbf{D}_f\mathbf{y}_f = \begin{bmatrix} -1 \\ -1 \end{bmatrix}$$

which gives

$$\mathbf{x} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} -0.5 \\ -0.5 \end{bmatrix}$$

So both free nodes settle to the same height and align vertically in the middle of the span.

In [17]:

```
# Recover full geometry, branch lengths, and branch forces

coords = {
    1: np.array([0.0, 0.0]),
    2: np.array([6.0, 0.0]),
    3: np.array([x[0], y[0]]),
    4: np.array([x[1], y[1]]),
}

lengths = []
forces = []

for qi, (j, k) in zip(q, branches):
    vec = coords[j] - coords[k]
    li = np.linalg.norm(vec)
    si = qi * li
    lengths.append(li)
    forces.append(si)

lengths = np.array(lengths)
forces = np.array(forces)

print("Branch lengths l =\n", lengths)
print("\nBranch forces s = L q =\n", forces)
```

```
Branch lengths l =
 [3.476 6.    2.634 1.212 1.738]

Branch forces s = L q =
 [3.476 6.    2.634 1.212 0.869]
```

In [18]:

```
# Plot the resulting network

fig, ax = plt.subplots(figsize=(7, 4.5))

for (j, k), qi in zip(branches, q):
    xvals = [coords[j][0], coords[k][0]]
    yvals = [coords[j][1], coords[k][1]]
    ax.plot(xvals, yvals, linewidth=2)
    xm = 0.5 * (coords[j][0] + coords[k][0])
    ym = 0.5 * (coords[j][1] + coords[k][1])
    ax.text(xm, ym + 0.08, f"q={qi}", ha="center")

for node, xy in coords.items():
    ax.scatter(xy[0], xy[1], s=60)
    ax.text(xy[0] + 0.08, xy[1] + 0.08, f"{node}")

ax.axhline(0, linewidth=0.8)
ax.set_aspect("equal", adjustable="box")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Force-Density Form-Finding Example")
plt.show()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[18], line 3
      1 # Plot the resulting network
----> 3 fig, ax = plt.subplots(figsize=(7, 4.5))
      5 for (j, k), qi in zip(branches, q):
      6     xvals = [coords[j][0], coords[k][0]]

NameError: name 'plt' is not defined
```

interpretation of the result

A few important observations:

- the method did **not** require us to guess the final shape and iterate to equilibrium
- the prescribed force densities controlled the relative stiffness of the branches in a geometric sense
- once the coordinates are known, member forces follow from

$$s_i = q_i l_i$$

This also shows one limitation of the classical method:

If the chosen force densities are not physically meaningful for the structure we actually want, the resulting shape may not be useful.

So force density is elegant, but it places a lot of modeling importance on the selection of q_i.

Part 6 - Back to the Unified Framework

> *connecting Schek to Veenendaal and Block*

where force density sits among other methods

In the Veenendaal and Block comparison, force density is one member of a family of form-finding procedures.

Very loosely:

- **force density** uses prescribed $q_i = s_i/l_i$ and solves directly for geometry
- **dynamic relaxation** introduces pseudo-dynamics and converges iteratively to equilibrium
- **stiffness method** uses element constitutive behavior and solves a nonlinear equilibrium problem

So these methods differ in:

- which variables are treated as primary
- whether the update is direct or iterative
- how the internal forces are related to the geometry

The force density method is especially attractive because it turns equilibrium into a **linear algebra problem** at the nodal level.

relation to what we have done in this course

There is also a useful conceptual bridge to the direct stiffness method:

In DSM for trusses, equilibrium is typically written in terms of:

- nodal displacements
- element axial deformations
- element axial forces from constitutive behavior

In force density, the formulation is more geometric:

- unknowns are directly the node coordinates
- connectivity is encoded by $mathbfC$
- the "material-like" parameter is the prescribed force density q_i

So although the algebra looks different, both approaches are still enforcing **nodal equilibrium of an interconnected bar or cable network**.

That is exactly why the Veenendaal and Block paper is helpful: it clarifies how methods that look very different are actually closely related.

Wrap-Up

In this lecture, we:

- introduced the force density method as a classical form-finding method
- defined the branch-node matrix and its partition into free and fixed parts
- derived the linear equilibrium equations
- solved a small planar example using prescribed force densities
- placed the method inside the broader framework of Veenendaal and Block

Next, we can build on this by comparing force density more directly with dynamic relaxation and stiffness-based form-finding, and by discussing how the choice of force densities influences the final shape.